Abhijatri

A Bangla-First, Offline-Ready Community Travel Platform for Bangladesh

---

Project Proposal

CSE 3102: Software Development Project

Team	ChoturMatra
Project type	Software development
Platform	Bangla-first, offline-ready Progressive Web Application (PWA)
Proposal date	12 October 2026
Revised	8 November 2026
Final completion	17 or 18 January 2027

Team Members

Responsibility	Member	Primary contribution
Project leader	Shahriar Islam	Scope control, content, estimator rules, administration, integration, and documentation
Frontend and offline	Abdullah Bin Sorwar Choudhury	React, TypeScript, bilingual interface, PWA, and offline packs
Backend and data	Syed Muhtasim Alam	FastAPI, PostgreSQL, authentication, APIs, rating, and idempotency
Software quality assurance	Md. Irfan Iqbal	Test strategy, validation, real-device testing, pilot, and regression

Contents

1 Executive Summary **1**

2 Problem Statement **2**

2.1 Fragmented travel information 2

2.2 Unclear trip costs 2

2.3 Weak connectivity 2

2.4 Limited Bangla-first support 2

2.5 Trust, moderation, and freshness 2

3 Objectives and Goals **3**

4 Functional Requirements **3**

4.1 Core Product Features 3

4.2 Future Product Features 4

4.3 User Roles 5

4.4 Functional Requirement Catalogue 6

4.5 Abhijatri Cost Planner 7

4.6 Primary Use Cases 10

4.7 Representative User Stories 10

4.8 MVP Exclusions 11

5 Non-functional Requirements **12**

6 Comparison with Existing Applications **12**

7 Stakeholders **14**

8 Timeline and Milestones **14**

8.1 Formal Testing Phase 15

8.2 Progress Reporting 15

8.3 Project Gantt Chart 15

9 Risk Assessment **18**

10 Feasibility Study **19**

10.1 Technical Feasibility 19

10.2 Economic and Financial Feasibility 19

10.3 Schedule and Time Feasibility 19

10.4 Operational and Ethical Feasibility 20

10.5 Feasibility Conclusion 20

List of Figures

1 Abhijatri system use case model (UML). Packages 4 and 6c are demonstration-only. 11

2 Implementation Gantt chart: summary by phase and workstream (M1 to M4) 17

List of Tables

1 Project objectives and completion measures. 3

2 What each user role can do 5

3 Functional requirement catalogue 6

4 Cost planner inputs and their effect on the calculation 7

5 Quantity and period rules by pricing unit 8

6 Worked example of the cost calculation (illustrative prices, BDT). 9

7 Scope of the use case model 10

8 Representative user stories with requirement traceability 10

9 Non-functional requirements and acceptance measures 12

10 Travel features offered by Abhijatri and by existing kinds of application. 13

11 Stakeholders, interests, and engagement 14

12 Fixed milestones and acceptance evidence 14

13 Formal testing activities and required records 15

14 Schedule summary by phase 16

15 Primary-owner workload balance 16

16 Risk register with early warnings and mitigation 18

17 Semester cost allowance 19

1 Executive Summary

The problem. Travellers in Bangladesh often plan trips from social media posts, agency advertisements, personal blogs, maps, and informal recommendations. These sources vary in age, format, and reliability. They rarely explain why two trips to the same destination cost different amounts, they seldom say when a price or a restriction was last checked, and they become hard to use exactly where travellers need them most: in hill districts, rural areas, and remote destinations where mobile data is slow, unstable, or absent. International travel services offer reviews and maps, but not Bangla-first guidance with Bangladesh-specific pricing units, local seasonality, and contextual cost explanations in one place.

The product. Abhijatri is a community-driven travel platform designed for Bangladesh. It combines bilingual destination guidance, structured community reviews, transparent ratings, dated travel advisories, an explainable rule-based cost planner, and downloadable offline travel packs. Bangla is the default language; English is available through a persistent language control. Every piece of guidance shows its source and date, and expired information is never presented as current.

Scope decision. Abhijatri will be developed as a Bangla-first, offline-ready Progressive Web Application (PWA). The semester minimum viable product (MVP) is organized around one complete journey: a traveller explores a destination, generates a contextual cost estimate, downloads an offline travel pack, and submits a moderated review after reconnecting. Verified Member identity checks, group tours, tourmate matching, and tour chat are part of the platform. Vendor commerce and real payments are future phases. A reservation and payment sandbox may be demonstrated only as a non-production extension; it is not part of semester acceptance.

What the semester delivers. The outcome is a tested PWA with five seeded destination profiles: Cox's Bazar, Sajek, Sylhet, the Sundarbans, and Bandarban. Each profile carries bilingual guidance, dated sources, contacts, seasonality, and advisories with current, stale, and expired states. Reviews are structured, moderated before publication, and counted in a documented Bayesian rating; a review written without a connection stays on the device and is submitted after reconnecting without ever creating a duplicate. A downloaded travel pack opens in airplane mode, and payment credentials are never collected.

How it is built. The platform will not use machine learning for cost prediction. The cost planner uses documented pricing rules, source dates, pricing units, season factors, low–expected–high ranges, and confidence labels, so every estimate can be explained line by line. The application is a React and TypeScript PWA over a FastAPI and PostgreSQL backend, with role-based access, rate limits, and audit logs.

Trust and privacy. Community content can be duplicated, biased, or manipulated, so review counts, moderation status, source dates, confidence labels, and advisory validity stay visible, and readers can report reviews into a moderation queue with recorded reasons and an audit trail. Verified Member status is decided by a moderator from an identity document photo, a live selfie, and a verified phone number; the images are deleted after the decision and only the last four digits of the document number are kept. Sensitive credentials and identity records never enter offline packs.

Delivery. Team ChoturMatra (four members: project leadership, frontend and offline, backend and data, and software quality assurance) works to the milestones in the Timeline and Milestones section. Formal testing runs from 4 to 11 January 2027, and the final demonstration, covering the destination, estimate, offline, review, and moderation flow, completes the project on 17 or 18 January 2027.

2 Problem Statement

2.1 Fragmented travel information

Travel information for Bangladesh is distributed across many platforms. Users must compare content with different dates, assumptions, and commercial interests. Seasonal restrictions, permits, transport changes, verified contacts, and local guidance can be missed.

2.2 Unclear trip costs

A single average cannot represent a solo traveller, student, couple, family, or group. Accommodation can be priced per room, transport per person or per vehicle, meals per person, and guides per group or per day. A useful estimate must show the cost basis, the traveller context, and the source date.

2.3 Weak connectivity

Hill districts, rural areas, and remote destinations may have slow, unstable, or unavailable mobile data. Travellers can lose access to contacts, guidance, advisories, and cost details exactly when they need them. An online-only system does not meet this requirement.

2.4 Limited Bangla-first support

International travel services may provide reviews and maps, but they do not consistently provide Bangla-first guidance, Bangladesh-specific pricing units, local seasonality, and contextual cost explanations in one workflow.

2.5 Trust, moderation, and freshness

Community content can be duplicated, biased, or manipulated. Prices and travel restrictions also change. The system must show review counts, moderation status, source dates, confidence labels, and advisory validity. Expired information must not be presented as current.

2.6 Market Alternatives and Abhijatri's Gap Recovery

Popular travel applications solve important parts of the journey, but their centre of gravity is global mapping, review discovery, itinerary organisation, or trail navigation. Abhijatri does not claim to replace those specialist services. Its advantage is a single Bangladesh-focused workflow that combines Bangla-first guidance, explainable BDT cost ranges, source-dated advisories, moderated reviews, and a validated offline destination pack.

Table 1. Five established applications and the product gap addressed by Abhijatri

Application	Established strength	Gap recovered in Abhijatri
Google Maps	Place discovery, community reviews, directions, and downloadable map areas	Offline packs add Bangla guidance, emergency contacts, source-dated advisories, and itemised trip-cost context; stale or expired content is visibly flagged.
Tripadvisor	Large review corpus, comparison, live availability, and editable day-by-day trip suggestions	Bangladesh-specific pricing units and local seasonality feed a transparent low–expected–high BDT estimate instead of an unexplained generic budget.
Wanderlog	Collaborative itineraries, reservation import, budgeting, map view, and offline trip plans	Moderated local reviews, advisory-validity dates, bilingual destination content, and independently validated offline packs complement itinerary organisation.
TripIt	Converts bookings into one itinerary that remains accessible offline	Abhijatri begins before booking: it supports local discovery, contextual estimation, safety guidance, and community evidence even when no reservation email exists.
AllTrails	Trail discovery and premium offline maps, navigation, conditions, and wrong-turn alerts	Abhijatri covers complete destination travel—transport, food, accommodation, contacts, cost, reviews, and advisories—rather than trail navigation alone.

Competitive position. The recoverable gap is therefore not "more maps" or another booking catalogue. It is dependable local context: a traveller can explore a Bangladesh destination, understand how an estimate was calculated, download the supporting guidance before connectivity fails, and later submit a duplicate-safe moderated review. This integrated flow reduces the need to switch among several applications while keeping specialist navigation or booking services complementary.

Evidence basis (official product pages, accessed 9 November 2026): Google Maps documents offline driving-map limits; Tripadvisor documents review-grounded planning and live availability; Wanderlog documents itinerary, budgeting, reservation, and offline features; TripIt documents booking-based itinerary creation and offline access; AllTrails documents trail-focused offline navigation on paid tiers.

3 Objectives and Goals

Each objective is a product feature, measurable against the final completion date.

Table 1. Project objectives and completion measures

ID	Objective	Completion measure
O1	Deliver a bilingual PWA	Bangla-default interface, persistent English selection, and installable application shell
O2	Publish seeded destination profiles	Five profiles with bilingual guidance, dated sources, contacts, seasonality, and advisories
O3	Provide explainable cost estimation	Itemized BDT estimate with pricing units, low–expected–high range, per-person cost, source dates, and confidence label
O4	Provide dependable offline access	Every completed pack download opens successfully in 10 of 10 airplane-mode tests
O5	Implement trustworthy reviews	Structured reviews, offline drafts, moderation states, reporting, and duplicate-safe submission

4 Functional Requirements

4.1 Core Product Features

1. **Bangla-first destination guides.** Users can explore destination profiles covering attractions, accommodation, food, transport, local practices, contacts, seasonality, and dated advisories in Bangla and English.
2. **Structured community reviews.** Reviews include travel date, traveller type, comfort level, text, overall rating, and category ratings. Review counts and moderation status remain visible.
3. **Transparent destination rating.** The platform uses a documented Bayesian score based on published overall ratings and a fixed semester prior. Hidden, rejected, and deleted reviews are excluded by tested query logic.
4. **Context-based cost planner.** Users receive an itemized BDT estimate based on party size, traveller type, dates, duration, comfort tier, transport, accommodation, meals, and selected activities. The result separates shared and per-person costs (Section 4.5).
5. **Offline travel pack.** Users can download public destination guidance, contacts, coordinates, advisories, and the generated estimate. The pack remains available in airplane mode after a successful download.
6. **Saved trip brief.** Users can save an estimate, free-text notes, and optional local contacts. This is a compact trip summary, not a day-by-day itinerary planner.
7. **Offline review draft.** A review written without connectivity remains on the device and can be submitted after reconnection. Retry requests use an idempotency key to prevent duplicates.
8. **Advisory freshness.** Every advisory stores its source, verification date, effective date, and validity date. Current, stale, and expired states are presented distinctly.
9. **Lean moderation and administration.** Authorized users manage destinations, translations, prices, advisories, review decisions, reports, and audit records.
10. **Privacy-aware account access.** Email verification controls review submission. Verified Member status is approved by a moderator from an identity document photo, a live selfie and a verified phone

number; the images are deleted after the decision and only the last four digits of the document number are kept. Payment credentials are never collected.

11. **Find Tourmates.** Verified Members create tours with capacity, a recruitment deadline and an access policy (open, students, families, women-only, men-only, invite-only), approve join requests, and are confirmed automatically at the minimum group size. Compatibility matching, invitations, a shared workspace, native group chat after confirmation, safety tools, reporting and blocking are included.

4.2 Future Product Features

The following capabilities belong to the product vision but are not required for semester acceptance:

- **Abhijatri Perks:** clearly disclosed partner benefits from approved resorts, restaurants, transport providers, and guides.
- **Vendor participation:** vendor registration, partner verification, reservation requests, ticket requests, and commission-based agency participation.
- **Advanced verification and payments:** automated Election Commission NID checks, verified-visit badges, real payments, refunds, and live inventory confirmation.
- **Platform expansion:** native mobile applications, advanced route planning, and context-aware machine learning once sufficient verified data exists.

4.3 User Roles

Abhijatri has five roles. Guests, Members, and Verified Members use the public application; Moderators and Superadmins use a separate staff dashboard that ordinary users never see. Table 2 shows what each role can do.

Table 2. What each user role can do

Feature or action	Guest	Member	Verified Member	Mode-rator	Super-admin
Public application: reading and offline use					
Read destination guides, dated advisories, and emergency contacts in Bangla or English	✓	✓	✓	–	–
Save destination packs for offline use and read them in airplane mode	✓	✓	✓	–	–
Generate an itemized cost estimate	✓	✓	✓	–	–
Save a trip brief with notes and contacts	✗	✓	✓	–	–
Public application: reviews					
Write a review draft that stays on the device without a connection	✓	✓	✓	–	–
Submit a text review (moderated before publication)	✓	✓	✓	–	–
Check status, withdraw, appeal, or request removal with the receipt	✓	✓	✓	–	–
Report a published review	✓	✓	✓	–	–
Keep reviews in an account: “My reviews”, edit while pending, withdraw, appeal	✗	✓	✓	–	–
Rating counts towards the Community Rating (Bayesian score)	✗	✗	✓	–	–
Rating counts towards the separate Guest Feedback	✓	✗	✗	–	–
Verified Member badge on reviews	✗	✗	✓	–	–
Attach photos and a video to a review (checked before publication)	✗	✗	✓	–	–
Public application: account and community					
In-app notifications and a list of one’s own reports	✗	✓	✓	–	–
Request account deletion	✗	✓	✓	–	–
Apply for Verified Member status	✗	✓	–	–	–
Find tourmates: create or join tours, workspace, group chat (age 18 or over)	✗	✗	✓	–	–
Staff dashboard (two-factor sign-in)					
Work the moderation queue (reviews, reports, appeals) with a recorded reason	✗	✗	✗	✓	✓
Decide Verified Member applications	✗	✗	✗	✓	✓
Decide on review photos and video	✗	✗	✗	✓	✓
Open and reverse enforcement cases; search the audit trail; queue monitoring	✗	✗	✗	✓	✓
Manage destination content, prices, and advisories	✗	✗	✗	✓	✓

Table 2 (continued)

Feature or action	Guest	Member	Verified Member	Mode-rator	Super-admin
Grant and revoke the Moderator and Superadmin roles	✗	✗	✗	✗	✓
Manage notification delivery (templates, rules, suppressions)	✗	✗	✗	✗	✓

✓ allowed ✗ not allowed – not applicable: staff use the public application through their own Member account

4.4 Functional Requirement Catalogue

Table 3. Functional requirement catalogue

ID	Function	Input	Output or behaviour
FR01	Destination catalogue	Destination selection	Bilingual profile with attractions, transport, food, lodging, contacts, dates, and restrictions
FR02	Language selection	Bangla or English	Interface and available editorial content in the selected language
FR03	Account lifecycle	Email, password, and verification link	Verified session, reset, logout, and deletion request
FR04	Structured review	Rating, text, categories, and travel context	Pending review with a local draft retained until successful submission
FR05	Review moderation	Review and moderation decision	Published, hidden, rejected, or deleted state with actor, date, and reason
FR06	Review reporting	Published review and report reason	Moderator-visible report record with duplicate control
FR07	Destination rating	Eligible published overall ratings	Fixed-prior Bayesian score and visible review count
FR08	Cost planner	Party, traveller type, dates, comfort tier, transport, lodging, meals, and activities	Itemized BDT estimate with low, expected, and high values, per-person result, and confidence label
FR09	Pricing units	Source-dated price records	Calculation by person, room, vehicle, group, trip, day, night, meal, or entry
FR10	Offline pack	Destination and generated estimate	Versioned IndexedDB pack containing public guidance, contacts, coordinates, advisories, and costs
FR11	Saved trip brief	Estimate, notes, and optional contacts	Local record with a UUID and copy or delete action
FR12	Offline review draft	Review entered without connectivity	Local draft that survives refresh and session expiry
FR13	Retry-safe submission	Review and idempotency key	One server review for identical retries
FR14	Pack update	Existing pack and newer manifest	Atomic replacement after the new pack passes validation
FR15	Advisory freshness	Source and validity dates	Current, stale, or expired state presented to the user
FR16	Administration	Authorized management action	Updated public data and immutable audit event

Table 3 (continued)

ID	Function	Input	Output or behaviour
FR17	Storage control	Pack download or deletion request	Persistence status, storage notice, and local data removal

4.5 Abhijatri Cost Planner

Purpose. Travellers can find individual prices, but they rarely receive an estimate that explains how party size, travel dates, comfort tier, and shared costs change the total. The cost planner answers three practical questions before a trip: (i) can this group afford this trip, (ii) how should shared costs be divided fairly, and (iii) which choices (dates, comfort tier, transport) change the total most.

Approach. The MVP implements a deterministic rules engine. It does not use machine learning or probabilistic simulation, so every result can be reproduced by hand, reviewed, and tested. Each estimate is decision support, not a vendor quotation or a price guarantee.

Terminology. Two independent dimensions are used throughout. The **comfort tier** (economy, standard, or premium) is chosen by the user and selects which catalogue items apply. The **price range** (low, expected, or high) expresses the realistic spread of prices for the selected items within that tier.

4.5.1 Inputs and outputs

Table 4. Cost planner inputs and their effect on the calculation

Input	Values	Effect on the calculation
Destination and dates	Seeded destination; start and end date	Selects catalogue records and the season band of each chargeable day or night
Party	Party size n; traveller types (adult, student, child)	Sets the number of eligible travellers n_e for person-based items and the capacity division for shared items
Duration	Days d and nights m	Sets the number of chargeable periods
Comfort tier	Economy, standard, or premium	Selects the accommodation, transport, and meal records that apply
Choices	Transport mode, accommodation type, meal plan, activities	Includes or excludes cost categories and items

For every category the result shows the low, expected, and high cost, the pricing unit, the quantity used, the source date, and the confidence value. It also shows the group total, the per-person share, the split between shared and individual costs, the assumptions applied, and an overall confidence label.

4.5.2 Price catalogue

Each price record stores the destination, category, item, comfort tier, season applicability, pricing unit, capacity, low price, expected price, high price, source type, source reference, verification date, validity date, and confidence. Records must satisfy $p^L \leq p^E \leq p^H$, and room, vehicle, and group records must state a capacity. Official sources take priority for fixed charges such as entry fees and permits. Recent moderated community reports are used for variable categories such as accommodation, food, and local transport.

4.5.3 Deterministic calculation

For each included category i and each range value $v \in \{L, E, H\}$, the engine computes

$$K_i^v \;=\; p_i^v \times q_i \times \sum_{\tau \in T_i} s(\tau), \tag{1}$$

where p_i^v is the unit price for range value v, q_i is the number of units required, T_i is the set of chargeable periods, and $s(\tau)$ is the season factor of period τ. The quantity and period rules are fixed per pricing unit (Table 5). Season factors are maintained in configuration for regular, weekend, peak, and public-holiday periods; fixed official charges always use $s(\tau) = 1$. Summing over periods, rather than applying one factor to the whole trip, correctly prices a trip that mixes weekday and weekend nights.

Table 5. Quantity and period rules by pricing unit

Pricing unit	**Units required** q_i	**Chargeable periods** T_i	**Typical use**
Per person	n_e	One occurrence	Intercity bus fare
Per entry	n_e	One per visit	Park or site entry fee
Per room	$\lceil n/c_i \rceil$	Each night (m)	Hotel or resort room
Per vehicle	$\lceil n/c_i \rceil$	Each day used (d)	Local jeep, CNG, or boat
Per group	$\lceil n/c_i \rceil$	Each day or one trip	Guide or permit escort
Per meal	n_e	Meals per day $\times\, d$	Restaurant meals
Per day / night	1	Each day or night	Daily service charge
Per trip	1	One occurrence	Reserved transfer

Here c_i is the capacity of one room, vehicle, or group unit, and $\lceil \cdot \rceil$ denotes ceiling division. The group totals and the per-person share are

$$T^v = \sum_i K_i^v, \qquad P^v = \frac{T^v}{n}. \tag{2}$$

Shared categories (rooms, vehicles, group services) are divided by party size only for the per-person view, and the result lists shared and individual amounts separately so the group can settle costs fairly. Calculations use unrounded values; displayed amounts are rounded to the nearest BDT 10.

4.5.4 Worked example

Table 6 applies Equation (1) to four students travelling to Cox's Bazar for three days and two nights (Thursday to Saturday) at the standard comfort tier. The first night is a regular night ($s = 1.00$); the second night and the last two days fall on the weekend ($s = 1.15$). **All prices in this example are illustrative placeholders used to demonstrate the rules; they are not researched catalogue values.**

Unit prices used (low / expected / high): bus 2,000 / 2,400 / 3,000 per person; room 2,200 / 3,000 / 4,000 per night; meal 180 / 250 / 350; vehicle 900 / 1,200 / 1,600 per day; entry 50. For example, the expected room cost is $3{,}000 \times \lceil 4/2 \rceil \times (1.00 + 1.15) = 12{,}900$. The shared share of the expected total (room and vehicle) is BDT 16,860, and the individual share is BDT 18,800.

4.5.5 Confidence and freshness

Each category receives a confidence value $C_i \in [0, 1]$ computed from four documented components, each scaled to $[0, 1]$:

$$C_i = 0.35\,A_i + 0.25\,F_i + 0.20\,R_i + 0.20\,G_i, \tag{3}$$

where A_i is source authority (official 1.0, verified operator 0.8, moderated community report 0.6), F_i is freshness (1.0 within the category refresh interval and 0.5 when stale), $R_i = \min(k_i/3,\, 1)$ for k_i independent

Table 6. Worked example of the cost calculation (illustrative prices, BDT)

Category	Unit and rule	q_i	$\sum s(\tau)$	Low	Expected	High	C_i
Intercity bus (return)	Per person, fixed	4	1.00	8,000	9,600	12,000	0.90
Hotel room, capacity 2	Per room, 2 nights	2	2.15	9,460	12,900	17,200	0.70
Meals, 3 per day	Per meal, 9 meals	4	9.00	6,480	9,000	12,600	0.60
Local vehicle, capacity 4	Per vehicle, 3 days	1	3.30	2,970	3,960	5,280	0.50
Park entry (official)	Per entry, fixed	4	1.00	200	200	200	0.95
Group total T^v				**27,110**	**35,660**	**47,280**	
Per person P^v				**6,780**	**8,920**	**11,820**	

reports, and $G_i = \max\big(0,\ 1-(p_i^H - p_i^L)/(2\,p_i^E)\big)$ measures agreement between reports. Overall confidence is weighted by the expected cost of each category:

$$C_{\text{overall}} = \frac{\sum_i K_i^E C_i}{\sum_i K_i^E}. \tag{4}$$

A score of at least 0.80 is labelled **High**, a score from 0.50 to below 0.80 is **Medium**, and a lower score is **Low**; a Low result is shown with an explicit verification warning. In the worked example, $C_{\text{overall}} = 25{,}240/35{,}660 = 0.71$, which is Medium.

Freshness rules are applied before any calculation. Initial refresh intervals are 12 months for official fixed charges and 3 months for accommodation, food, and local transport. A record older than its refresh interval is **stale**: it may still be used, but with $F_i = 0.5$ and a visible warning. A record beyond its explicit validity date is **expired** and is excluded. The weights, factors, intervals, and thresholds above are initial configuration values; they are frozen with the rule specification (task E01), stored with a version number, and shown with every estimate.

4.5.6 Implementation and validation

The schedule follows the project Gantt chart (Section 8.3). The rule specification and worked examples are frozen first (E01). Source-dated price research covers four destinations during the second milestone (E02–E05), and the fifth destination is added during hardening (H01). The catalogue, rule engine, and result screens are then implemented (E06–E08), and the planner is tested against hand-calculated cases for every pricing unit, including odd party sizes, capacity one, stale records, expired records, and fixed charges that ignore season factors (E11–E12).

Ten independent trips are collected and sealed outside the seed catalogue. After the catalogue is frozen, the final report compares each expected estimate E with the actual in-scope cost A using the absolute percentage error $|E-A|/A$ per category and in total, the mean absolute percentage error (MAPE), and coverage, defined as the share of trips whose actual cost lies within $[T^L, T^H]$. The planner is compared with a simple baseline of one average per-person per-day cost multiplied by $n \times d$. The proposed acceptance target is a lower MAPE than the baseline and coverage of at least 7 of 10 trips. This scope is implementable with FastAPI, PostgreSQL, and standard arithmetic.

4.6 Primary Use Cases

The use case model (Figure 1) shows the complete Abhijatri PWA system boundary, its four actors, and the use cases grouped by functional domain. Table 7 summarises the scope of the diagram. Packages 4 (booking) and 6c (payment verification) are demonstration-only and lie outside semester acceptance, consistent with the scope decision. The editable source model is available online: **open the original use case diagram in Lucidchart**.

Table 7. Scope of the use case model

Element	Content
Actors	Guest, email-verified user, moderator, and administrator
Core packages	Supporting (language and account), 1. Cost Estimator, 2. Offline, 3. Review, 5. Community, Administration, 6a. Advisory and offline-pack verification, 6b. Email and account verification
Demonstration only	4. Booking (demo reservation) and 6c. Payment verification (demo). No money transfer and no live inventory
System boundary	Abhijatri PWA and its controlled management functions
Notation	Solid line: actor association. Dashed arrow labelled «include»: mandatory sub-behaviour. Dashed arrow labelled «extend»: optional or conditional behaviour. Bold ellipse: primary use case of a package

4.7 Representative User Stories

Table 8 lists representative user stories in the standard role–goal–benefit form, each traced to the functional requirements that satisfy it.

Table 8. Representative user stories with requirement traceability

ID	As a ...	I want ...	So that ...	Traces to
US1	Bangla-speaking traveller	destination guidance in Bangla	I can use the service without depending on English content	FR01, FR02
US2	Student travelling with friends	shared and per-person costs shown separately	we can divide the budget fairly	FR08, FR09
US3	Traveller entering a weak-network area	contacts, coordinates, advisories, and my estimate available offline	I can still act when the network fails	FR10, FR17
US4	Reviewer	my offline draft to survive session expiry	I can authenticate and submit without rewriting it	FR12, FR13
US5	Moderator	hidden and rejected reviews excluded from ratings	invalid content does not influence the score	FR05, FR07
US6	Administrator	stale prices and expired advisories identified	outdated data is never presented as current	FR15, FR16

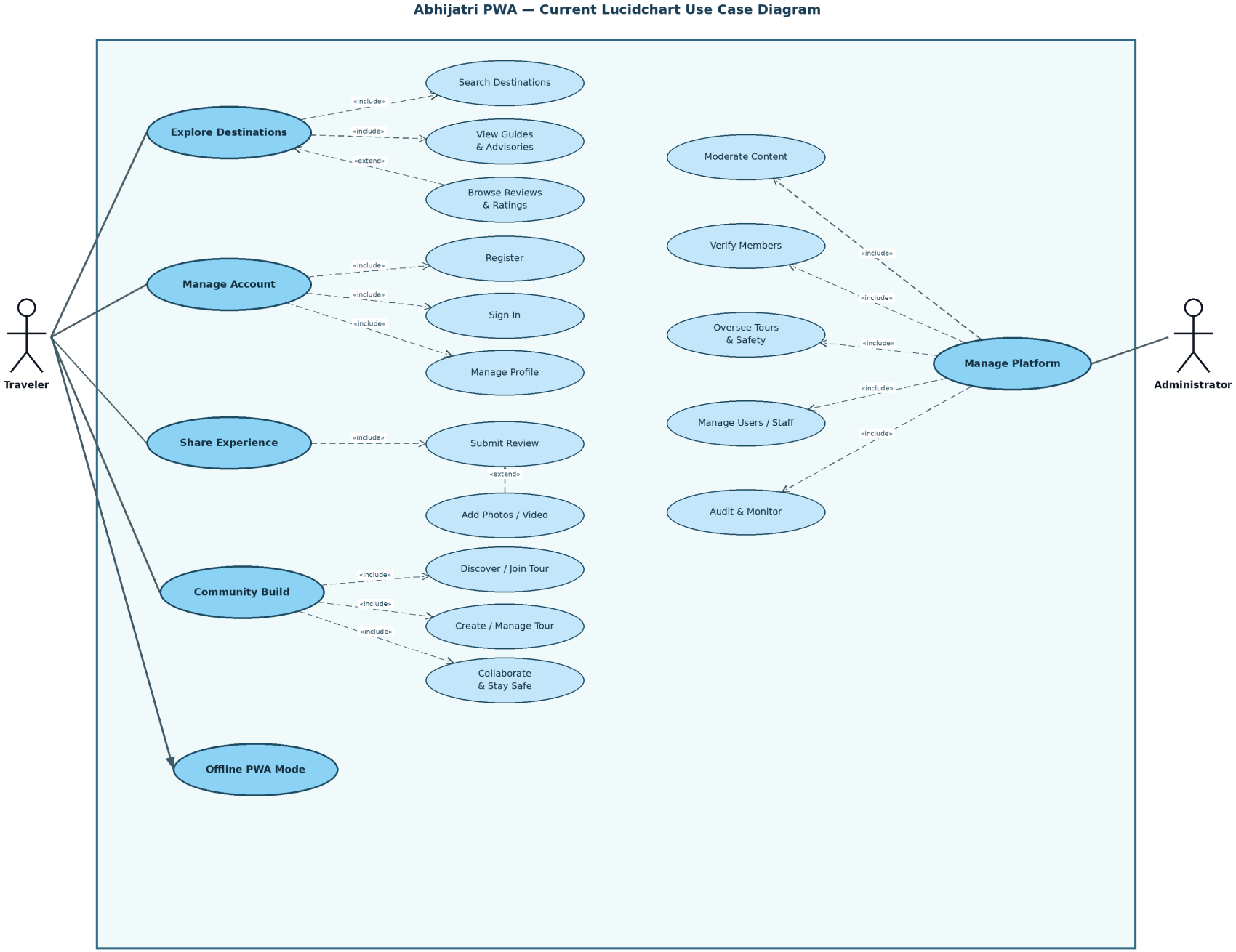


Figure 1. Abhijatri system use case model (UML). Updated from the current Lucidchart.

Editable source: current Abhijatri use case model in Lucidchart

4.8 MVP Exclusions

The semester acceptance scope excludes real payments, refunds, live inventory, vendor self-registration, verified-visit badges, traveller monitoring, day-by-day itinerary planning, route optimization, native mobile applications, automated translation, and machine-learning recommendations.

5 Non-functional Requirements

Table 9. Non-functional requirements and acceptance measures

Category	Requirement and acceptance measure
Performance	Core destination text becomes usable within 5 seconds under the agreed Fast 3G throttle on the target low-end Android device. A downloaded pack opens within 2 seconds. Catalogue, review, and estimator APIs remain below 2 seconds at the 95th percentile under a 50-user academic load test.
Security and access control	Argon2id password hashing, opaque server-side sessions, HttpOnly and Secure production cookies, CSRF protection, input validation, least-privilege roles, rate limits, and audit logs. Sensitive credentials and identity records never enter offline packs. As built, passwords and staff TOTP are held by Supabase Auth behind the API.
Reliability and availability	Identical retries do not create duplicate reviews. Interrupted pack updates preserve the last complete version. The deployed pilot includes a database backup and one restore test. Online, offline, stale, and expired states are distinct.
Scalability	Stateless FastAPI application layer, PostgreSQL indexes, pagination, and bounded queries. The architecture permits multiple API instances later. The semester load target is 50 concurrent virtual users.
Usability and accessibility	Bangla is the default locale. Forms have labels, keyboard access, clear errors, readable contrast, and tested Bengali conjunct rendering. Applicable WCAG 2.2 AA criteria are evaluated with at least eight users.
Maintainability	Modular React and FastAPI components, database migrations, API documentation, coding conventions, code review, and automated unit and integration tests. Each content category has an owner and an update interval.
Portability	Current Chromium-based Android browsers and modern desktop browsers. Safari limitations are documented and tested where a device is available.
Privacy	Minimum account and pilot data, informed consent, optional personal contacts kept local to the device, defined retention periods, and support for account deletion and review-removal requests.

6 Comparison with Existing Applications

Travellers in Bangladesh already combine several kinds of application, and each covers only part of what a trip needs. Table 10 shows which common travel features each kind typically offers. The columns describe the typical, publicly available capabilities of each kind of application in November 2026; individual applications and later versions may differ, and the examples are given only for orientation.

Table 10. Travel features offered by Abhijatri and by existing kinds of application

Feature	**Abhijatri**	**Global review sites** e.g. Tripadvisor	**Map apps** e.g. Google Maps	**Local booking apps** e.g. ShareTrip, GoZayaan	**Social media groups** e.g. Facebook groups
Destination and attraction information	✓	✓	✓	✗	✓
Traveller reviews and ratings	✓	✓	✓	✓	✗
Bangla-language interface	✓	✗	✓	✗	✓
Works without an internet connection	✓	✗	✓	✗	✗
Trip price information	✓	✓	✗	✓	✗
Group tours or finding travel companions	✓	✗	✗	✓	✓
Dated safety advisories with emergency contacts	✓	✗	✗	✗	✗
Explainable cost estimate (pricing units, low–expected–high range, source dates)	✓	✗	✗	✗	✗
Features covered	**8 of 8**	3 of 8	4 of 8	3 of 8	3 of 8

✓ offered ✗ not offered as a typical capability of that kind of application

Each existing kind of application offers some of these features, but none offers them all, so a traveller today moves between several applications and still has no dated safety information or explainable cost estimate. Abhijatri brings all of them into one Bangla-first application that keeps working without a connection.

7 Stakeholders

Table 11. Stakeholders, interests, and engagement

Stakeholder	Interest or authority	Engagement
Course instructor and evaluators	Approve scope, review progress, and evaluate the outcome	Proposal, biweekly reports, demonstrations, and final evidence
Travellers and tourists	Use destination, estimate, offline, and review functions	Usability sessions and pilot feedback
Local residents and experienced travellers	Provide local context and cost reports	Consent-based pilot and source verification
Moderators and administrators	Protect content quality and maintain dated information	Moderation and administration tests
Destination authorities	Provide restrictions, permits, and safety notices	Dated official public sources
Future vendors and agencies	Provide services and future commercial participation	Future onboarding and commission policy

8 Timeline and Milestones

The schedule follows the fixed dates agreed by the team. Early completion is allowed, but acceptance dates are not moved without instructor approval.

Table 12. Fixed milestones and acceptance evidence

Date	Gate	Milestone	Acceptance evidence
2 November 2026	M1	Major feature accepted	Destination information, structured reviews, bilingual interface, moderation, and offline support operate as one flow
7 December 2026	M2	Significant progress	Approximately 50 to 60 percent of the fixed MVP is accepted, including estimator integration and evidence
3 January 2027	G1	Testing-entry gate	Frozen build, test data, requirement traceability, environment, and procedures are approved
4–11 January 2027	M3	Formal testing	Full-flow, non-functional, usability, regression, defect, and exit records are complete
17 or 18 January 2027	M4	Final completion	Release candidate, report, evidence archive, deployment notes, and demonstration are complete

Table 13 (continued)

Area	Schedule	Activities and required records

8.1 Formal Testing Phase

Table 13. Formal testing activities and required records

Area	Schedule	Activities and required records
Test leadership	Before execution	Irfan leads the master test plan, test-case register, execution record, and exit recommendation. Every requirement is mapped to test IDs.
Test-case record	4–11 January	Record ID, requirement, preconditions, data, steps, expected result, actual result, status, evidence, defect, severity, owner, and retest result.
Destination and estimate	4–11 January	Browse a seeded destination, switch language, read dated guidance, and generate context-specific estimates.
Offline flow	4–11 January	Save a trip brief, download a pack, use airplane mode, reopen essential information, reconnect, and verify refresh status.
Review and moderation	4–11 January	Submit an offline draft, retry safely, moderate it, and verify rating exclusions.
Access and failures	4–11 January	Test roles, pricing units, stale warnings, duplicate requests, network failures, and review reporting.
Non-functional testing	4–11 January	Test throttled 2G and 3G, airplane mode, low-end Android, Bangla rendering, accessibility, storage refusal, cache updates, privacy, and response times.
Usability	4–11 January	Test at least eight consenting users, including a Bangla-preferring and a less technical participant.
Defect management	4–11 January	Record reproducible evidence, assign severity and owner, retest fixes, and run affected-flow regression.
Exit criteria	By 11 January	Every in-scope requirement has a status. No blocker or critical defect remains in the core flow. Residual lower-severity issues are documented.

8.2 Progress Reporting

Each biweekly report records planned work, completed work, evidence, measured progress, defects, blocked items, changed assumptions, accepted work, and approved scope changes.

8.3 Project Gantt Chart

The operational schedule contains 74 tasks in six phases, with dependencies, primary owners, effort points, and five fixed gates. Table 14 summarises the phases, Table 15 shows that primary effort is balanced at 87 points (25 percent) per member, and Figure 2 summarises the chart by phase and workstream. Acceptance evidence and risk notes for every task are maintained in the live schedule: **open the live 74-task implementation schedule in Google Sheets**.

Reading the chart. Each row groups related tasks, with its task IDs; a dot marks every week in which that work is active, with week columns labelled by the date of each Monday. Coloured diamonds and vertical dashed lines mark the fixed dates M1 (2 Nov), M2 (7 Dec), G1 (3 Jan), M3 (11 Jan), and M4 (17 Jan). Individual tasks, owners, dates, dependencies, and acceptance evidence are in the live schedule.

Table 14. Schedule summary by phase

Phase	Window	Tasks	Effort	Exit gate
Phase 1: Foundation	12–16 Oct 2026	7	29	Scope, architecture, test plan
Phase 2: Destination, reviews, offline	15 Oct – 2 Nov 2026	21	99	M1, 2 Nov 2026
Phase 3: Cost estimator and checkpoint	3 Nov – 7 Dec 2026	15	75	M2, 7 Dec 2026
Phase 4: Hardening and test readiness	8 Dec 2026 – 3 Jan 2027	12	60	G1, 3 Jan 2027
Phase 5: Formal testing	4–11 Jan 2027	10	48	M3, 11 Jan 2027
Phase 6: Completion and final delivery	12–18 Jan 2027	9	37	M4, 17/18 Jan 2027
Total	**14 weeks**	**74**	**348**	

Table 15. Primary-owner workload balance

Owner	Role	Tasks	Effort	Share
Shahriar Islam	Project leader, content, moderation	22	87	25%
Abdullah Bin Sorwar Choudhury	Frontend, bilingual interface, offline	17	87	25%
Syed Muhtasim Alam	Backend, data, security	17	87	25%
Md. Irfan Iqbal	SQA, test management	18	87	25%
Total		**74**	**348**	**100%**

Figure 2. Implementation Gantt chart: summary by phase and workstream (M1 to M4)
Task-level schedule: **open the Gantt chart in Google Sheets**

IDs	Work grouped by workstream	October 2026			November				December					January 2027		
		12	19	26	2	9	16	23	30	7	14	21	28	4	11	
Phase 1: Foundation																
F01–F07	Scope, architecture and API contract, UX plan, test plan, content template, repositories and CI, security checklist	●														
Phase 2: Destination, reviews, offline																
R01–R11	Destination and account services and screens, review rules, services and screens, rating, moderation and reporting	●	●	●												
R12–R16	Installable offline app, offline destination packs, pack API, offline review drafts, idempotency		●	●												
R17–R20	Functional and offline testing, integration and defect triage, M1 acceptance regression		●	●	●											
M1	**Major feature accepted: destination + reviews + bilingual offline support**				◆ M1											
Phase 3: Cost estimator and checkpoint																
E01–E05	Estimator rules, pricing units and confidence; price and advisory research for four destinations				●	●										
E06–E10	Pricing catalogue, estimator engine and screens, saved trip brief, price and advisory maintenance					●	●	●	●							
E11–E14	Estimator test scenarios and tests, integration of all flows, progress audit and evidence pack					●	●	●	●							
M2	**Original core scope progress checkpoint: 50–60% complete**									◆ M2						
Phase 4: Hardening and test readiness																
H01–H11	Remaining destinations, review UX, offline and backend reliability, security, moderation polish, regression, pilot, accessibility, release readiness									●	●	●	●			
G1	**Testing-entry readiness review**													◆ G1		
Phase 5: Formal testing																
T01–T09	Test build and records, functional and non-functional testing, defect fixing, regression, exit summary													●	●	
M3	**Formal testing completed**														◆ M3	
Phase 6: Completion and final delivery																
C01–C08	Defect closure, content freeze, release smoke test, documentation, final demonstration and demo data														●	
M4	**Final project completion**														M4 ◆	

9 Risk Assessment

Table 16. Risk register with early warnings and mitigation

Risk	Level	Early warning	Mitigation and owner
Scope growth	High	New features requested during a cycle	Freeze the semester capabilities. Any addition replaces work of equal effort. Owner: Shahriar.
Insufficient destination or price data	High	Missing sources or conflicting values	Begin research in the first cycle. Store provenance, date, unit, tier, and confidence. Owners: all content owners.
Weak estimator accuracy	High	Large error on development cases	Use ranges, transparent rules, a baseline, and sealed held-out trips. Owners: Shahriar and Irfan.
Stale safety information	High	Advisory lacks a current official source	Store verification and validity dates. Recheck before publication and demonstrations. Owners: destination owners.
Offline storage eviction	Medium	Persistence is refused or a pack disappears	Show status, re-download, and delete controls. Test on real devices. Owners: Abdullah and Irfan.
Authentication delay	Medium	Account API misses the planned cycle	Continue the guest destination and pack slice. Complete accounts early in the next cycle. Owner: Muhtasim.
Team unavailability	High	Missed review or unmerged work	Maintain shared documentation, small reviewed changes, tests, and a backup owner for each module. Owner: Shahriar.
Privacy or security defect	High	Sensitive data appears in logs, cache, or pack	Apply data minimisation, network-only private APIs, security tests, and a release gate. Owners: Muhtasim and Irfan.
Bangla rendering failure	Medium	Broken conjuncts or unreadable wrapping	Use system-font fallback and a device-version test matrix. Owner: Abdullah.
Low pilot participation	Medium	Fewer than eight usable participants	Recruit early and maintain backup participants. Owner: Irfan.
Copyright issue	Medium	Image or source has unclear permission	Use owned or openly licensed images and record licences. Owner: each content owner.
Deadline pressure	High	The 7 December target or testing entry is at risk	Review scope at the checkpoint and protect the fixed testing and final dates. Owners: Shahriar and team.
Third-party service change	Medium	A free tier ends, pauses, or hits its limit (Supabase, Upstash, Vercel, email provider)	Keep the database portable, keep two email providers configured, and keep a paid fallback inside the BDT 6,000 ceiling. Owner: Muhtasim.

10 Feasibility Study

10.1 Technical Feasibility

The selected technologies match the team roles. React and TypeScript support the bilingual frontend. FastAPI supports the API layer. PostgreSQL provides authoritative relational storage. Workbox manages the versioned application shell. IndexedDB through Dexie stores downloaded packs, saved trip briefs, and review drafts. Authenticated API responses are excluded from public service-worker caches.

The MVP avoids dependencies on real payments, automated NID verification, live inventory, and native application packaging; phone verification uses one SMS gateway behind a replaceable interface. The highest-risk capability, offline access, is tested with one complete destination before the first milestone. The architecture is therefore achievable by the team within the semester.

10.2 Economic and Financial Feasibility

The MVP uses open-source frameworks and existing student hardware. The figures below are planning allowances, not quotations.

Table 17. Semester cost allowance

Cost category	Estimated BDT	Basis
Development frameworks and database	0	Open-source software
Existing laptops and Android test phones	0	Team-owned equipment
Test connectivity, calls, and data collection	1,500	Pilot and source-verification allowance
Optional domain and short-term hosting	2,500	Used only if an external demonstration requires it
Contingency	2,000	Unexpected testing or deployment expense
Planning ceiling	**6,000**	Maximum proposed semester allowance

The future business model can include vendor subscriptions, booking commissions, clearly labelled sponsorship, and optional premium services. Sponsorship must not affect ratings or moderation.

10.3 Schedule and Time Feasibility

The Gantt chart assigns dependencies, evidence, risk notes, and effort to each task, and workload is balanced at 87 primary effort points per member. The first complete vertical slice is due on 2 November. The 7 December checkpoint leaves time to reduce destination quantity before reducing quality. Formal testing has a protected eight-day window, followed by a final correction and release period.

If work falls behind, the team will remove future-phase functions before changing core acceptance criteria. Real booking, real payment, and automated NID verification remain outside the semester dependency chain.

10.4 Operational and Ethical Feasibility

Pilot participation is voluntary and uses a short consent notice. The project does not store NID numbers, payment credentials, or unnecessary personal documents; identity images for verification are deleted after the moderator's decision. Optional contacts remain local to the user's device. Abhijatri will not present itself as an emergency-response or traveller-monitoring service. Restrictions and advisories must be verified again before publication or demonstration.

10.5 Feasibility Conclusion

Abhijatri is feasible as a semester project if the team preserves the MVP boundary and treats commercial and social expansion as future work. The selected technology is available, direct academic cost is limited, risk is tested early, and the schedule includes formal acceptance evidence.